
Two-Level Autoresearch: Optimizing LLM Policy Synthesis in Sequential Social Dilemmas

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We demonstrate an autonomous AI agent that discovers effective configurations
2 for a complex multi-agent system—without human intervention. A *researcher*
3 *agent* $\mathcal{R}$ (Claude Opus 4.6) iteratively modifies the synthesis pipeline—system
4 prompts, feedback construction, helper libraries, iteration logic—under which a
5 *policy-synthesizer* LLM $\mathcal{M}$ generates cooperative strategies for Sequential Social
6 Dilemmas. The researcher reads code, edits source files, runs evaluations, and
7 decides what to try next, following the *autoresearch* paradigm. We conduct 12
8 autonomous runs across two games (Cleanup and Gathering), two policy LLMs
9 (Gemini 3.1 Pro and Claude Sonnet 4.6), and two social welfare objectives (utilitar-
10 ian efficiency and Rawlsian maximin). The researcher reliably exceeds baselines:
11 in Cleanup, efficiency improves by 66% (Gemini) and 263% (Sonnet), while dra-
12 matically tightening run-to-run spread (max/mean gap: 0.05 for Gemini and 0.02
13 for Sonnet, vs. baseline ranges of ~ 1.5). Most strikingly, maximin-optimized
14 pipelines achieve near-perfect equality ($E=0.98$) with negligible efficiency loss
15 (-1%), discovering duty rotation as the key fairness mechanism—never found un-
16 der efficiency optimization. The full-pipeline approach substantially outperforms
17 prompt-only optimization (GEPA), achieving $3\times$ higher efficiency in Cleanup and
18 37% higher in Gathering, as prompt-level search cannot overcome the high vari-
19 ance of LLM code generation. The contrast with Gathering, where efficiency op-
20 timization alone yields $E>0.94$, reveals that game structure determines whether
21 fairness requires explicit optimization: in provision dilemmas fairness needs de-
22 signed mechanisms, while in restraint dilemmas it emerges as a byproduct of co-
23 ordination.

24 1 Background

25 We build on the iterative LLM policy synthesis framework of Gallego [1]. An SSD is a partially
26 observable Markov game $\mathcal{G} = \langle N, \mathcal{S}, \{A_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, H \rangle$. Social outcomes are evaluated via
27 four metrics [2]: efficiency U , equality E , sustainability S , and peace P . We additionally consider
28 the *maximin* (Rawlsian) welfare criterion $\min_i R_i$, which maximizes the return of the worst-off
29 agent.

30 We study two SSDs that represent complementary dilemma types. **Cleanup** [3] is a *public goods*
31 *provision* game ($N=10$): a river accumulates waste while an orchard grows apples. Cleaning costs
32 -1 per beam but enables apple regrowth for everyone ($+1$ per apple). The dilemma: free-ride on
33 cleaning vs. contribute. **Gathering** [4, 2] is a *common pool resource* game ($N=4$): agents collect
34 shared apples and may tag rivals to temporarily remove them. The dilemma: over-harvest and
35 aggress vs. cooperate and share. The games differ in their cost structure—asymmetric provision vs.
36 symmetric restraint—a distinction that drives our key findings.

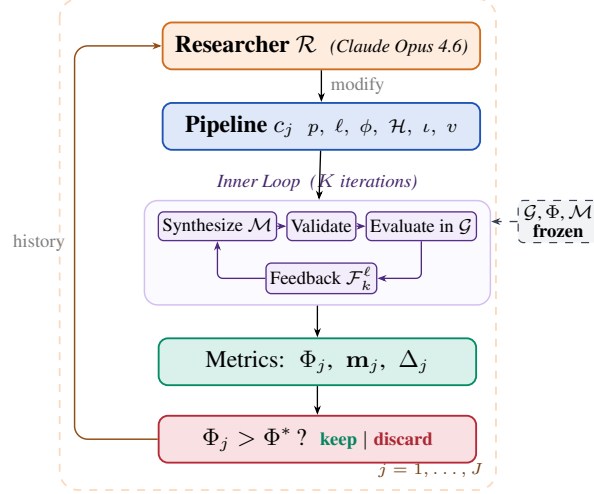


Figure 1: Two-level automated research framework (Algorithm 1). *Outer loop*: the researcher $\mathcal{R}$ modifies the pipeline configuration c_j , runs the inner loop, and keeps or discards based on Φ . *Inner loop*: the policy LLM $\mathcal{M}$ iteratively synthesizes, validates, and evaluates policies in the frozen environment $\mathcal{G}$.

37 A frozen LLM $\mathcal{M}$ acts as a policy synthesizer. Given a system prompt p and a feedback prompt
38 q_k , it generates a Python policy $\pi_{k+1} = \mathcal{M}(p, q(\pi_k, \mathcal{F}_k^\ell))$, where ℓ specifies the feedback level
39 and $\mathcal{F}_k = (\bar{r}_k, \mathbf{m}_k)$ contains the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k =$
40 (U_k, E_k, S_k, P_k) . All N agents execute the same policy (homogeneous self-play). Gallego [1]
41 compared two hand-designed feedback levels—sparse (reward-only) and dense (reward + social
42 metrics)—and showed that dense feedback consistently improves cooperative outcomes. The dense
43 feedback configuration achieves $U=1.93$ (mean) / 2.70 (max) with Gemini and $U=0.86$ / 1.56 with
44 Sonnet on Cleanup (4 runs each).

45 2 Two-Level Framework

46 We introduce a two-level system where a *researcher agent* $\mathcal{R}$ autonomously discovers configurations
47 that optimize the output of an inner-loop system. While we instantiate this for multi-agent policy
48 synthesis, the architecture is general: any pipeline where an LLM generates artifacts, evaluates them,
49 and iterates can serve as the inner loop. The key insight is that the entire inner-loop codebase—not
50 just the feedback level—is a designable artifact that an AI agent can search over. Figure 1 illustrates
51 the architecture.

52 2.1 Configuration Space

53 Let $\mathcal{C}$ denote the space of *pipeline configurations*. Each configuration $c \in \mathcal{C}$ specifies the full inner-
54 loop setup:

$$c = (p, \ell, \phi, \mathcal{H}, \iota, v) \quad (1)$$

55 where p is the system prompt, ℓ is the feedback template, ϕ is the feedback construction function,
56 $\mathcal{H}$ is the helper function library, ι specifies the iteration logic (number of iterations K , sampling
57 strategy, temperature schedule), and v is the validation pipeline. Table 1 provides concrete examples.

58 The sparse vs. dense comparison of [1] corresponds to a single binary choice within the ℓ component.
59 Our framework opens the full configuration space to automated search.

60 2.2 Inner Loop (Policy Synthesis)

61 Given a configuration c , the inner loop executes K iterations of LLM policy synthesis:

$$\pi_c^* = \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c) \quad (2)$$

62 Each iteration k proceeds in four stages, following [1]:

Table 1: Configuration components modifiable by the researcher. The environment simulator, ground-truth evaluation, and policy LLM weights are frozen.

	Scope	Examples
p	System prompt	Hints, worked examples
ℓ	Feedback template	Metric subset, framing
ϕ	Feedback fn	Derived metrics, trends
$\mathcal{H}$	Helper library	Pathfinding, zones
ι	Iteration logic	K , eval seeds, best-of- n
v	Validation	Safety checks, smoke tests

Algorithm 1 Two-Level Automated Research

Require: Game $\mathcal{G}$, policy synthesizer $\mathcal{M}$, researcher $\mathcal{R}$, system prompt for researcher $p_{\mathcal{R}}$, initial configuration c_0 , outer iterations J , ground-truth objective Φ , held-out seeds $S_{\text{held-out}}$

Ensure: Best configuration c^* , best policy π^*

```

1:  $\pi_0^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_0)$ 
2:  $\Phi_0 \leftarrow \text{EVAL}(\pi_0^*, \mathcal{G}, S_{\text{held-out}})$ 
3:  $history \leftarrow \{(c_0, \Phi_0, \mathbf{m}_0, \emptyset)\}$ 
4: for  $j = 1, \dots, J$  do
5:    $c_j \leftarrow \mathcal{R}(p_{\mathcal{R}}, \text{CODE}(c_{j-1}), history)$ 
6:   if  $\neg \text{VALIDATECONFIG}(c_j)$  then  $\text{retry} \leq R$  times
7:      $\pi_j^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_j)$ 
8:      $\Phi_j \leftarrow \text{EVAL}(\pi_j^*, \mathcal{G}, S_{\text{held-out}})$ 
9:      $\Delta_j \leftarrow \text{DIFF}(c_{j-1}, c_j)$  // code diff
10:     $history \leftarrow history \cup \{(c_j, \Phi_j, \mathbf{m}_j, \Delta_j)\}$ 
11: end for
12:  $j^* \leftarrow \arg \max_j \Phi_j$ 
13: return  $c_{j^*}, \pi_{j^*}^*$ 

```

- 63 1. **Synthesize.** The policy LLM $\mathcal{M}$ receives the system prompt p , the previous policy’s source
64 code π_{k-1} , and feedback $\mathcal{F}_{k-1}^\ell$ constructed according to (ℓ, ϕ) . It generates a new Python policy
65 function π_k that has access to full environment state and the helper library $\mathcal{H}$.
- 66 2. **Validate.** The generated code undergoes AST-based safety checks (blocking dangerous oper-
67 ations such as file I/O and network access) followed by a short smoke test. Failures trigger
68 re-generation (up to R retries), with the error message appended to the prompt.
- 69 3. **Evaluate.** All N agents execute the same policy π_k in self-play over $|S|$ random seeds. The eval-
70 uation yields the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$.
- 71 4. **Feedback.** The feedback function ϕ constructs the prompt for the next iteration from
72 $(\pi_k, \bar{r}_k, \mathbf{m}_k)$ according to the template ℓ . Under sparse feedback, $\mathcal{M}$ sees only the scalar reward;
73 under dense feedback, it additionally receives all social metrics with natural-language definitions.
- 74 The inner loop output is evaluated on held-out seeds via a fixed ground-truth objective:

$$\Phi(c) = \text{EVAL}(\pi_c^*; \mathcal{G}, S_{\text{held-out}}) \quad (3)$$

75 where Φ maps from social outcomes to a scalar. We consider two objectives:

$$\Phi_U = U \quad (\text{utilitarian efficiency}) \quad (4)$$

$$\Phi_{\min} = \min_i R_i \quad (\text{Rawlsian maximin}) \quad (5)$$

76 2.3 Outer Loop (Automated Research)

77 The researcher agent $\mathcal{R}$ iteratively modifies the pipeline configuration. Following the autoresearch
78 paradigm [5], $\mathcal{R}$ operates on the inner-loop codebase as a modifiable artifact, proposing changes,
79 observing outcomes, and refining. The procedure is formalized in Algorithm 1.

80 At each outer iteration j , the researcher $\mathcal{R}$ receives:

- 81 • The **full source code** of the current configuration c_{j-1} (prompts, feedback construction, helpers,
82 iteration logic).

Table 2: Structural analogy between autoresearch [5] for LLM pretraining and our framework for multi-agent policy synthesis.

	Autoresearch	This work
Modifiable artifact	<code>train.py</code> (model + optimizer)	Inner-loop codebase ($p, \ell, \phi, \mathcal{H}, \iota, v$)
Frozen infrastructure	<code>prepare.py</code> (data, tokenizer, eval)	Environment $\mathcal{G}$, eval Φ , LLM $\mathcal{M}$
Metric	<code>val_bpb</code> ↓	Φ (social welfare) ↑
Budget	5 min wall clock	K iters $\times$ $ S $ seeds
Agent	AI coding agent	Researcher $\mathcal{R}$

- The **experiment history**: for each prior iteration, the code diff Δ_i , ground-truth score Φ_i , and social metrics vector $\mathbf{m}_i$.
- The **environment source code** (read-only), enabling the researcher to reason about game mechanics.

The researcher proposes a new configuration c_j by generating code modifications. Concretely, $\mathcal{R}$ is a coding agent (Claude Code CLI) that operates on a real software repository: it reads and edits Python source files, runs shell commands, inspects evaluation outputs, and commits changes to a dedicated git branch—the same workflow a human researcher would follow. This mirrors the autoresearch setup, where an AI coding agent modifies `train.py` and observes validation loss; here, the researcher modifies the inner-loop codebase and observes social welfare.

2.4 Structural Analogy

Table 2 formalizes the correspondence between autoresearch for LLM pretraining and our two-level system.

The key structural property shared by both systems: the agent modifies *how learning happens* (the pipeline/training code), not the learned artifact directly (the policy/model weights). The output of each outer iteration is a *reusable configuration*—transferable across policy LLMs, games, and random seeds—rather than a single policy.

2.5 Design Principles

Separation of concerns. The researcher $\mathcal{R}$ and the synthesizer $\mathcal{M}$ are distinct LLM calls with distinct roles. $\mathcal{R}$ reasons about *what information and tools help $\mathcal{M}$ write better policies*; $\mathcal{M}$ reasons about *what policy to write given its current context*. Using different underlying models ($\mathcal{R} \neq \mathcal{M}$) avoids conflation and enables cross-model experiments.

Read-only environment. The researcher can read the environment source code to understand game mechanics but cannot modify the simulator. This prevents the trivial solution of making the game easier and cleanly separates mechanism design from environment design.

Fixed compute budget. Each inner loop runs for exactly K iterations with $|S|$ seeds. The researcher cannot inflate performance by running more iterations, but can change what happens *within* those K iterations (e.g., best-of- n sampling, varying the prompt structure mid-run).

Diff-based history. The researcher sees code diffs Δ_j from previous experiments alongside their outcomes. This enables causal reasoning about which modifications drove which improvements, mirroring autoresearch’s experiment logs.

3 Connection to Automated Mechanism Design

The two-level structure admits a mechanism design interpretation. The researcher $\mathcal{R}$ acts as a *mechanism designer*: it controls the information structure (what metrics to reveal, how to frame them),

the action space (what helper functions are available), and the incentive structure (how feedback is presented) under which the policy synthesizer $\mathcal{M}$ operates. The synthesizer acts as the *agent* within the designed mechanism.

This connects to the automated mechanism design literature [6], where a principal designs rules to induce desired behavior from self-interested agents. In our setting:

- The **principal** is the researcher $\mathcal{R}$, optimizing the ground-truth objective Φ .
- The **agent** is the synthesizer $\mathcal{M}$, optimizing per-agent reward as instructed.
- The **mechanism** is the configuration c : prompts, feedback, helpers, iteration logic.
- The **outcome** is the social welfare of the resulting multi-agent policy π_c^* .

A key distinction from classical mechanism design: $\mathcal{M}$ is not *strategically* self-interested—it follows instructions—but it has *bounded rationality* in the sense that its ability to synthesize effective policies depends on the information and tools provided. The researcher’s task is thus closer to *information design* [7]: choosing what to reveal to help $\mathcal{M}$ navigate the cooperation–defection tension. Our experiments (Section 4) show that the researcher designs qualitatively different information structures depending on the welfare objective Φ , supporting this interpretation empirically.

4 Experiments

4.1 Setup

We conduct 12 autonomous researcher runs across a factorial design. The researcher agent $\mathcal{R}$ is Claude Opus 4.6, invoked via the Claude Code CLI as a coding agent. Each run operates on a dedicated git branch of a real Python codebase: $\mathcal{R}$ edits source files in `pipeline/`, executes evaluation scripts, reads metric outputs, and iterates—without human intervention.

Design. For Cleanup: 2 policy LLMs $\times$ 2 objectives $\times$ 2 replications = 8 runs. For Gathering: 2 policy LLMs $\times$ 1 objective $\times$ 2 replications = 4 runs. Maximin runs are unnecessary for Gathering because efficiency optimization alone achieves equality $E > 0.94$ (the game’s symmetric cost structure makes fairness a free byproduct of coordination).

Models. Policy synthesizer $\mathcal{M}$: Gemini 3.1 Pro (Google) or Claude Sonnet 4.6 (Anthropic). Both use extended thinking. These are the same models evaluated in [1]. We additionally test with Gemma 4 26B-A4B-IT (Google), a smaller open-weight model, to probe the framework’s behavior when the policy synthesizer has substantially lower capability (Appendix C).

Baselines. The hand-designed dense feedback (reward + social metrics) configuration from [1] serves as the initial pipeline c_0 for all runs. On Cleanup ($N=10$), this baseline achieves $U=1.93/2.70$ (mean/max) with Gemini and $U=0.86/1.56$ with Sonnet across 4 runs each. We additionally compare against GEPA [8], an automated prompt optimization method that iteratively refines the system prompt via LLM reflection. GEPA optimizes only the system prompt p , whereas our researcher modifies the full pipeline $(p, \ell, \phi, \mathcal{H}, \iota, v)$. We give GEPA a matched compute budget: the same number of optimization steps as outer iterations used by our method (each GEPA step costs ~ 3 policy evaluations, comparable to our inner loop’s $K+1=4$).

Inner loop parameters. $K=3$ synthesis iterations (or $K=2$ if the researcher elects), evaluated over $|S|=5$ –12 random seeds (researcher-configurable). The researcher ran $J=3$ –17 outer iterations per run (mean: 8.3), with a keep rate of 18%–100% (mean: 63%).

4.2 Results

Table 3 presents the main results, comparing autoresearch against the hand-designed baseline of [1] and GEPA prompt optimization [8].

Finding 1: The researcher reliably improves over hand-designed baselines and outperforms prompt-only optimization. All 12 runs show substantial improvement regardless of starting point (Figure 2). In Cleanup, the researcher exceeds the unmodified pipeline baseline by 66% with Gemini ($U: 1.93 \rightarrow 3.20$) and 263% with Sonnet ($U: 0.86 \rightarrow 3.12$). Strikingly, the Sonnet improvement

Table 3: Results. Mean / max across 2 runs per condition (4 for Cleanup baselines). Baseline: unmodified pipeline with dense feedback from [1]. GEPA: automated prompt optimization [8] with matched compute budget.

Policy LLM	Target	U	E	$\min_i R_i$
<i>Cleanup (N=10) — Baseline</i>				
Gemini 3.1 Pro	—	1.93/2.70	0.17/0.62	−159/−84
Sonnet 4.6	—	0.86/1.56	−0.02/0.25	−151/−59
<i>Cleanup (N=10) — GEPA [8]</i>				
Gemini 3.1 Pro	Φ_U	1.34/1.37	−0.10/−0.05	−149/−126
	$\Phi_{\min}$	1.76/2.76	0.90/0.96	143/245
Sonnet 4.6	Φ_U	1.04/1.20	−0.59/−0.21	−164/−126
	$\Phi_{\min}$	−0.63/1.60	0.77/1.00	−147/6
<i>Cleanup (N=10) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	3.20 /3.25	0.55/0.61	−211/−182
	$\Phi_{\min}$	3.16/3.19	0.98 /0.98	290 /296
Sonnet 4.6	Φ_U	3.12 /3.14	0.66/0.70	−196/−146
	$\Phi_{\min}$	2.57/2.93	0.91 /0.97	179 /200
<i>Gathering (N=4) — Baseline</i>				
Gemini 3.1 Pro	—	2.04/2.42	0.90/0.98	412/571
Sonnet 4.6	—	0.03/0.03	0.54/0.54	0/0
<i>Gathering (N=4) — GEPA [8]</i>				
Gemini 3.1 Pro	Φ_U	2.08/2.35	0.94/0.96	436/518
Sonnet 4.6	Φ_U	1.20/1.23	0.63/0.71	86/123
<i>Gathering (N=4) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	2.49/2.51	0.98/0.98	582/582
Sonnet 4.6	Φ_U	2.52/2.52	0.96/0.98	576/597

164 nearly closes the gap with Gemini ($U=3.12$ vs. 3.20), despite a $2\times$ baseline disadvantage (0.86 vs.
165 1.93). Final efficiency values cluster tightly around $U \approx 3.1$ –3.2 despite baselines varying from
166 0.29 to 2.70 across individual runs, suggesting the researcher reliably discovers the performance
167 ceiling of each policy LLM. Beyond improving the mean, the researcher yields remarkably consistent
168 results: under efficiency optimization, Gemini’s runs cluster within $U=3.20$ –3.25 (gap 0.05)
169 despite baselines spanning 1.15–2.70, and Sonnet’s within 3.12–3.14 (gap 0.02) from baselines
170 of 0.38–1.56. In Gathering, all four runs converge to $U=2.47$ –2.52 from baselines ranging 0.03–
171 2.42. The researcher discovers pipeline modifications—structured helpers (App. D.1), strategic hints
172 (App. D.2), tailored feedback (App. D.3)—that constrain the policy LLM toward consistently good
173 solutions.

174 Compared to GEPA (Table 3), autoresearch achieves $2.4\times$ higher efficiency under Φ_U in Cleanup
175 ($U=3.20$ vs. 1.34), $2\times$ higher maximin under $\Phi_{\min}$ (290 vs. 143), and 20% higher efficiency in Gath-
176 ering ($U=2.49$ vs. 2.08). Critically, GEPA shows far wider spread across runs on the harder fairness
177 objective ($\Phi_{\min}$ maximin in Cleanup: 245/41 vs. autoresearch’s 296/284). The gap widens further
178 with Sonnet: on Cleanup Φ_U , GEPA stalls at $U=1.04$ ($3\times$ below autoresearch’s 3.12), and on $\Phi_{\min}$
179 its two runs split between a modestly positive solution ($\min_i R_i=6$, $U=1.60$) and a pathological
180 “everyone cleans, nobody eats” regime (perfect equality $E=1.00$ but negative efficiency $U=-2.87$
181 and $\min_i R_i=-299$), while autoresearch with the same model reaches $\min_i R_i=179/200$ reliably.
182 This highlights the advantage of modifying the full pipeline—helpers, feedback, and iteration logic
183 provide structured scaffolding that reduces run-to-run variance, whereas prompt-only optimization
184 cannot address the underlying stochasticity of code generation.

185 **Finding 2: No efficiency–fairness tradeoff in Cleanup (Gemini).** Maximin-optimized Gem-
186 ini pipelines sacrifice only 1% efficiency (U : 3.16 vs. 3.20) while achieving near-perfect equal-
187 ity (E : 0.98 vs. 0.55) and transforming maximin from deeply negative baselines (−99 to −84) to
188 $\min_i R_i = 290$. The researcher discovers *fair duty rotation* (Listing 6; primed by the prompt rewrite
189 of Listing 3)—time-based cycling using `agent_id` and `env._step_count`—simultaneously im-
190 proving worst-off welfare *and* collective output. Because cleaning is a public good, distributing the
191 cleaning cost fairly ensures enough cleaners to sustain apple production.

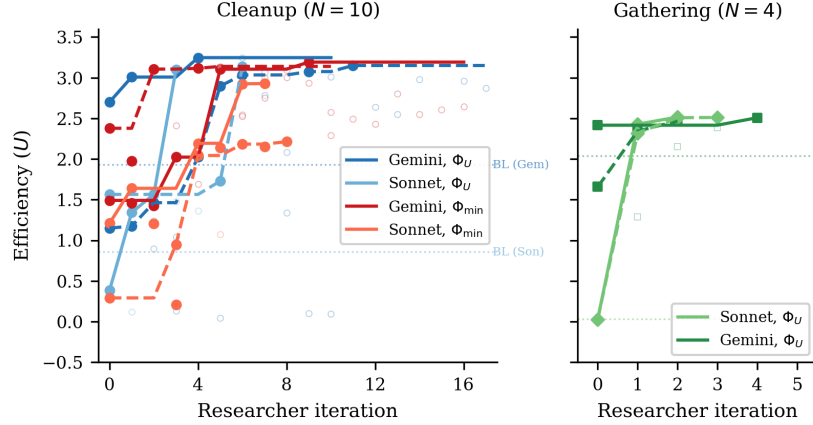


Figure 2: Efficiency (U) across researcher iterations for all 12 runs. *Left*: Cleanup ($N=10$) with 8 runs across 2 LLMs $\times$ 2 objectives (solid lines = kept experiments, open circles = discarded). Dashed horizontal line: hand-designed baseline from [1]. All runs converge to $U \approx 3.1$ – 3.2 despite diverse starting points. *Right*: Gathering ($N=4$) with 4 runs (2 per LLM). All converge to $U \approx 2.5$ within 2–4 iterations despite baselines ranging from 0.03 to 2.42.

For Sonnet, there is a moderate tradeoff: efficiency drops from 3.12 to 2.57 under maximin optimization. The gap reflects Sonnet’s harder time implementing complex coordination mechanisms (role rotation, zone assignment) from strategic hints alone.

Finding 3: Game structure determines whether fairness requires explicit optimization. In Cleanup, where cleaning costs are borne asymmetrically (cleaners pay -1 , free-riders collect apples), baseline equality ranges from $E=0.04$ to 0.62 , and maximin optimization is required to reach $E > 0.9$. In Gathering, where all agents face a symmetric landscape, efficiency optimization alone achieves $E > 0.94$ across all 4 runs—no separate maximin runs are needed.

This generalizes: in *provision* dilemmas with asymmetric costs, fairness requires designed mechanisms (role rotation, duty sharing); in *restraint* dilemmas with symmetric costs, fairness emerges as a free byproduct of efficient coordination. The researcher independently discovers this—it creates role differentiation pipelines only for Cleanup, and pure spatial-coordination pipelines for Gathering.

Finding 4: Convergent discovery of qualitatively different strategies per objective. Despite fully independent runs, the researcher converges on the same core strategies within each condition (Table 4, Appendix A). In Cleanup, waste-counting helpers and spatial zone partitioning appear across all 8 runs. The key qualitative difference is *role rotation*, appearing in 4/4 maximin runs (Listings 6, 7) but 0/4 efficiency runs (Listing 5). Under efficiency optimization, the researcher assigns static cleaning roles (some agents always clean), achieving high collective output at the cost of equality. Under maximin optimization, it discovers rotation as a fairness mechanism.

In Gathering, the researcher discovers BFS-Voronoi territory partitioning and respawn-timer awareness (Listings 2, 8), with no role differentiation—optimization is purely spatial (who collects which apples) and temporal (respawn-aware positioning).

Common failure modes. Several patterns led to discarded experiments across all runs: (1) *over-prescription*—adding too many strategic hints confuses the policy LLM, causing generation failures; (2) *iteration regression*— $K=4$ or $K=5$ inner iterations often cause catastrophic regression as the LLM “over-refines” a working policy; (3) *feedback overload*—dense per-agent diagnostics cause over-correction rather than gradual improvement; (4) *variance exposure*—identical configurations can yield $U=3.2$ then $U=0.09$ on different seeds, requiring multi-seed evaluation for stable signals (per-run K , $|S|$ in Table 7).

5 Discussion and Conclusion

We have presented a two-level framework where an AI coding agent autonomously discovers pipeline configurations that improve the output of an inner-loop LLM system. Applied to multi-agent policy synthesis in social dilemmas, the researcher agent reliably exceeds hand-designed baselines across 12 independent runs, and converges on qualitatively similar strategies within each game-objective condition. The framework requires no domain-specific scaffolding: the agent operates on a standard software repository using the same tools (file editing, shell commands, git) available to a human researcher.

Mechanism design in action. Our results empirically support the mechanism design interpretation of Section 3. The researcher acts as an information designer: under Φ_U , it reveals efficiency-oriented information (waste counts, zone assignments) that guides $\mathcal{M}$ toward productive but unequal role allocation (Listing 5). Under $\Phi_{\min}$, it additionally reveals fairness-oriented structure—rotation schedules and per-agent equity feedback (Listings 3, 4)—guiding $\mathcal{M}$ toward egalitarian coordination. The striking absence of a fairness–efficiency tradeoff with Gemini suggests that in public goods provision, the mechanism design problem has a cooperative optimum where both objectives align—the researcher discovers this independently.

Two types of dilemma. The Cleanup–Gathering contrast reveals a structural insight about social dilemmas: *provision* dilemmas (asymmetric costs, one agent’s contribution benefits all) require explicit fairness mechanisms, while *restraint* dilemmas (symmetric costs, each agent faces the same temptation) achieve fairness naturally when coordination improves. This distinction, well-known in the common-pool resource literature, is rediscovered here by an autonomous AI agent—suggesting it is a robust structural feature of these environments.

Human oversight and audit. Our system is fully autonomous by design, but its architecture offers natural affordances for human-in-the-loop oversight. Because the researcher $\mathcal{R}$ operates on a standard git repository—committing each pipeline modification to a dedicated branch—a domain expert can audit the full history of code diffs Δ_j alongside their metric outcomes $(\Phi_j, \mathbf{m}_j)$, exactly as they would review a colleague’s pull requests. The keep/discard decision (Algorithm 1, line 11) is a lightweight intervention point: a human could override the accept threshold, veto modifications that introduce undesirable mechanisms (e.g., exploitative role assignments), or steer the researcher toward under-explored regions of the configuration space by editing the experiment history. More broadly, the separation between the researcher $\mathcal{R}$ and the frozen evaluation Φ creates a *delegation boundary*: the human defines *what* to optimize (the welfare objective), while the agent decides *how*. This mirrors the expert-in-the-loop design patterns identified in deployment-oriented work on AI agents for discovery, where trust calibration depends on the legibility of the agent’s decision trail rather than on real-time supervision.

Limitations. Our experiments use only 2 replications per condition, yielding high uncertainty on some estimates (e.g., Sonnet $\Phi_{\min}$: $U=2.57/2.93$). The two SSDs are small gridworld environments; scaling to larger, more complex games remains open. The researcher runs are not fully controlled—the number of outer iterations varies from 3 to 17 (mean 8.3)—making direct comparison across conditions approximate. The GEPA comparison uses a single run per condition; additional replications would strengthen confidence in the comparison.

Future work. Five directions emerge: (1) *new domains*—apply the two-level framework to other LLM-driven pipelines (code optimization, scientific experiment design, infrastructure tuning), where the same architecture applies with a different inner loop and objective Φ ; (2) *ablation*—freeze all configuration components except one to isolate the relative importance of prompt, feedback, helpers, and iteration logic; (3) *adversarial objectives*—set $\Phi = R_0$ (agent 0’s reward only) to test whether the researcher discovers exploitative pipeline configurations, connecting to the reward hacking analysis of [1]; (4) *cross-game transfer*—run the best Cleanup configuration on Gathering and vice versa, testing game-generalizability; (5) *heterogeneous policies*—extend to settings where agents run different code, enabling richer role specialization. Code is available at <https://github.com/vicgalle/llm-policies-social-dilemmas>.

References

- [1] Víctor Gallego. Cooperation and exploitation in LLM policy synthesis for sequential social dilemmas. In *Proc. Adaptive and Learning Agents Workshop (ALA 2026)*, 2026.
- [2] Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Graepel. A multi-agent reinforcement learning model of common-pool resource appropriation. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [3] Edward Hughes, Joel Z Leibo, Matthew Phillips, Karl Tuyls, Edgar A Dueñez-Guzmán, Antonio García Castañeda, Iain Dunning, Tina Zhu, Kevin R McKee, R. Koster, Heather Roff, and Thore Graepel. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Neural Information Processing Systems*, 2018.
- [4] Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Janssen, and Thore Graepel. Multi-agent reinforcement learning in sequential social dilemmas. In *Proc. 16th Conference on Autonomous Agents and MultiAgent Systems*, pages 464–473, 2017.
- [5] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026.
- [6] Vincent Conitzer and Tuomas Sandholm. Complexity of mechanism design. In *Proc. 18th Conference on Uncertainty in Artificial Intelligence*, pages 103–110, 2002.
- [7] Emir Kamenica and Matthew Gentzkow. Bayesian persuasion. *American Economic Review*, 101(6):2590–2615, 2011.
- [8] Lakshya A Agrawal et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2026.

A Additional Results

Table 4: Strategies discovered by the researcher in Cleanup across 4 efficiency-optimized and 4 maximin-optimized independent runs.

Strategy	Φ_U (4 runs)	$\Phi_{\min}$ (4 runs)
Waste-counting helpers	4/4	4/4
Zone/lane partitioning	3/4	4/4
Anti-regression feedback	3/4	2/4
Worked policy examples	2/4	3/4
Cleaning cost economics	2/4	3/4
Time-based role rotation	0/4	4/4

B Compute Requirements

Table 5 reports wall-clock time for all 12 autonomous runs and the 6 GEPA comparison runs. *Inner loop* measures total time spent on policy LLM generation and environment simulation across all evaluations; *total* (estimated from run-directory timestamps) additionally includes the researcher agent’s analysis, code editing, and decision-making between evaluations.

Cost breakdown. Each inner loop evaluation involves $K=2\text{--}3$ policy LLM generation calls (each $\sim 2\text{k}$ input tokens, $\sim 1.5\text{k}$ output tokens) followed by multi-seed simulation (5–12 seeds, $\sim 15\text{--}30\text{s}$ total). Policy LLM generation dominates inner loop time (86–97%), with Sonnet evaluations taking $\sim 2\times$ longer than Gemini due to extended thinking. The researcher agent (Claude Opus 4.6, running via Claude Code CLI) accounts for 41% of total wall-clock time (25.6h of the 62.2h total across all 12 runs), spent reading results, editing pipeline source files, and planning modifications. In monetary terms, the researcher agent is the dominant cost: each outer iteration consumes $\sim 50\text{--}100\text{k}$ context tokens in the Opus session, whereas each inner loop evaluation uses only $\sim 5\text{--}10\text{k}$ policy LLM tokens. The 6 GEPA comparison runs required 5.0h of compute with no researcher agent, operating at lower cost per run but achieving substantially lower performance (Table 3).

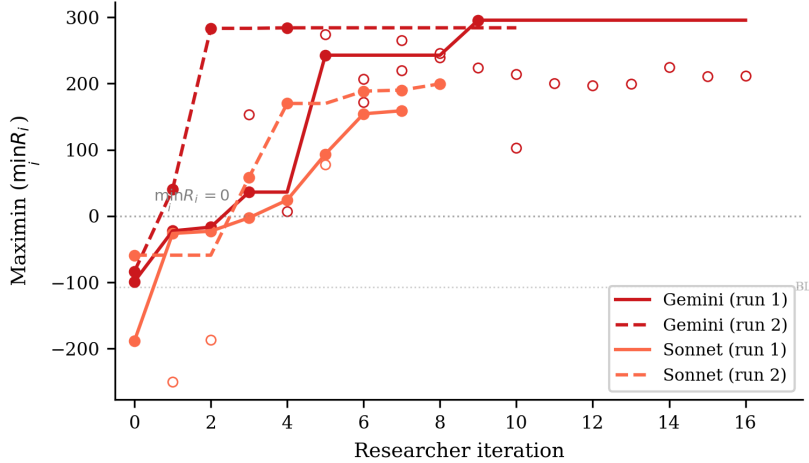


Figure 3: Maximin ($\min_i R_i$) across researcher iterations for the 4 Cleanup maximin-optimized runs. All runs transform deeply negative baselines (worst-off agents losing reward) into substantially positive values. Gemini reaches ~ 290 while Sonnet reaches ~ 160 – 200 . The dashed line marks $\min_i R_i = 0$ (no agent loses reward overall).

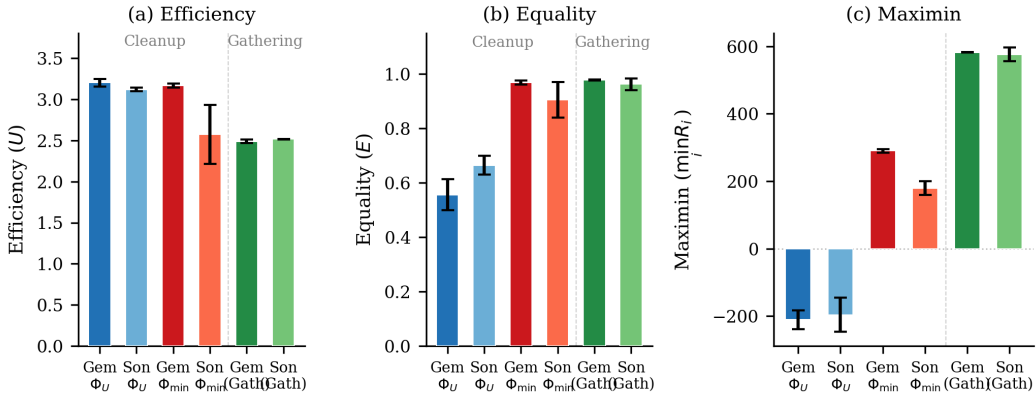


Figure 4: Final metrics across all conditions. (a) Efficiency: all conditions converge to $U \approx 2.5$ – 3.2 . (b) Equality: maximin-optimized Cleanup runs achieve $E \approx 1.0$, while efficiency-optimized runs show $E \approx 0.5$ – 0.7 ; Gathering achieves high equality regardless. (c) Maximin: the sharpest contrast—efficiency optimization leaves the worst-off agent deeply negative ($\min_i R_i \approx -200$), while maximin optimization transforms it to $+290$ (Gemini) or $+179$ (Sonnet). Gathering achieves high maximin (~ 580) under efficiency optimization alone. Error bars show s.d. across runs.

310 C Smaller Open-Weight Model: Gemma 4 26B

311 We test the framework with Gemma 4 26B-A4B-IT (Google), a 26B-parameter open-weight model
 312 substantially smaller than the frontier models in the main experiments. We run three experiments—
 313 two on Cleanup ($N=10$) optimizing efficiency and maximin respectively, and one on Gathering
 314 ($N=4$) optimizing efficiency—all with Opus 4.6 as the researcher $\mathcal{R}$.

315 **Setup.** In all three experiments, Gemma starts from complete failure: the baseline pipeline pro-
 316 duces $U=-10.0$ on Cleanup (agents CLEAN-spam without collecting apples) and $U=0.0$ on Gath-
 317 ering (broken BFS calls), compared to $U=1.93/0.86$ (Gemini/Sonnet on Cleanup) and $U=2.04/0.03$
 318 (Gemini/Sonnet on Gathering). The researcher ran $J=10$ – 18 outer iterations per condition.

Table 5: Wall-clock time per autonomous run. Evals: total inner loop evaluations including initial baseline. Per eval: mean wall-clock per single evaluation. Total: end-to-end including researcher overhead.

Policy LLM	Φ	Evals	Inner loop (h)	Per eval (min)	Total (h)
<i>Cleanup</i> ($N=10$)					
Gemini	Φ_U	11	2.6	14	3.7
Gemini	Φ_U	18	4.1	14	6.4
Sonnet	Φ_U	4	2.1	32	6.1
Sonnet	Φ_U	7	5.0	43	6.1
Gemini	$\Phi_{\min}$	17	3.0	11	4.3
Gemini	$\Phi_{\min}$	11	3.6	20	5.0
Sonnet	$\Phi_{\min}$	8	4.5	33	8.8
Sonnet	$\Phi_{\min}$	9	5.3	36	13.2
<i>Gathering</i> ($N=4$)					
Sonnet	Φ_U	3	1.7	33	2.2
Gemini	Φ_U	5	1.4	17	1.9
Gemini	Φ_U	3	0.9	19	1.1
Sonnet	Φ_U	4	2.3	35	3.4
All 12 runs		100	36.6	22	62.2
GEPA (6 runs)		6	5.0	50	—

Results. Table 6 presents the best configurations discovered. Under efficiency optimization, the researcher achieves $U=0.87$ —a dramatic recovery from the broken baseline but far below frontier models ($U \approx 3.2$). The researcher compensates for the model’s limited capability by reducing inner-loop iterations to $K=1$ (avoiding the regression that plagued $K \geq 2$ runs) and providing highly structured worked examples with explicit cleaning-role assignment.

Table 6: Autoresearch with Gemma 4 26B-A4B-IT. Single run per condition. Baseline: dense feedback pipeline from [1].

Game	Target	U	E	$\min_i R_i$	Keep rate
Cleanup	Baseline	−10.0	1.00 [†]	−1000	—
	Φ_U	0.87	−1.11	−371	6/19
	$\Phi_{\min}$	1.71	0.94	137	9/17
Gathering	Baseline	0.0	1.00 [†]	0	—
	Φ_U	2.44	0.98	580	4/11

[†]Equality is trivially 1.0 because all agents receive near-zero reward.

Maximin optimization rescues efficiency. Strikingly, under maximin optimization the researcher achieves *higher* efficiency ($U=1.71$) than under direct efficiency optimization ($U=0.87$), alongside strong equality ($E=0.94$) and positive worst-off welfare ($\min_i R_i=137$). This reversal—absent in frontier models, where both objectives yield $U \approx 3.1$ – 3.2 —occurs because the maximin objective forces the researcher toward coordination mechanisms (rotating cleaning duties, index-based apple assignment) that simultaneously improve collective output. Under efficiency-only optimization, the researcher converges on a local optimum—static role assignment with 25% dedicated cleaners—that a 26B model can implement but that caps performance well below the frontier.

This suggests that *for models below a capability threshold, fairness objectives may serve as better optimization targets for overall social welfare than direct efficiency maximization*. The structured coordination enforced by the maximin objective provides a scaffold that compensates for the weaker model’s difficulty implementing complex strategies from strategic hints alone.

Gathering: full recovery on a simpler game. On Gathering ($N=4$), the researcher brings Gemma from $U=0.0$ to $U=2.44$ with $E=0.98$ and $\min_i R_i=580$, after 10 outer iterations (4 kept). These values nearly match frontier models (Gemini: $U=2.49$, Sonnet: $U=2.52$; Table 3). The researcher discovers the same Voronoi partitioning and respawn-aware camping strategies found

in the main experiments, and increases inner-loop iterations to $K=5$ to allow sufficient refinement. The contrast with Cleanup—where Gemma peaks at $U=0.87/1.71$ versus frontier $U \approx 3.2$ —suggests that the researcher can fully compensate for model weakness on simpler coordination tasks (4 agents, symmetric costs) but not on harder provision dilemmas (10 agents, asymmetric costs).

D Researcher-Authored Pipeline Artifacts

The previous appendix shows the policies π_c^* that the inner loop *outputs*; this one shows the configuration $c = (p, \ell, \phi, \mathcal{H}, \iota, v)$ (Section 4, Table 1) that the researcher $\mathcal{R}$ *authored* to produce them. Excerpts are taken from the final commit on the dedicated git branch of the corresponding run; we reproduce the prose verbatim, with author commentary in [brackets] and elisions marked We organize by artifact type so each subsection makes a within-type contrast (e.g., Φ_U vs. $\Phi_{\min}$, or Cleanup vs. Gathering).

D.1 Helper library $\mathcal{H}$: coordination primitives

The researcher adds primitives that the policy LLM can call as black boxes, sparing it from re-implementing tricky logic on every iteration. Two patterns dominate: (i) *state inspection* (waste counts, fractions, beam-yield scoring) and (ii) *coordination primitives* (zone assignment, role rotation). We show one of each.

Listing 1: Cleanup, $\Phi_{\min}$ – band-based apple zoning helper added by the researcher in exp5. This is the primitive that lets the policy in Listing 6 keep collectors within their own row band, preventing all 7 gatherers from racing to the same apple.

```

356 1 def get_my_apples(env, agent_id):
357 2     """Alive apples in this agent's horizontal row band.
358 3     Divides the orchard into n_agents bands; falls back to all apples if empty."""
359 4     aid = int(agent_id)
360 5     n = int(env.n_agents)
361 6     band_h = env.height / n
362 7     band_lo, band_hi = aid * band_h, (aid + 1) * band_h
363 8
364 9     my_apples, all_apples = set(), set()
365 10    for i in range(env.n_apples):
366 11        if env.apple_alive[i]:
367 12            ar = int(env._apple_pos[i, 0]); ac = int(env._apple_pos[i, 1])
368 13            all_apples.add((ar, ac))
369 14            if band_lo <= ar < band_hi:
370 15                my_apples.add((ar, ac))
371 16    return my_apples if my_apples else all_apples
372

```

Listing 2: Gathering – BFS-Voronoi territory + respawn-aware waiting helpers added by the researcher in gather-exp3. These are the primitives that Listing 8 composes into a one-line policy: “go to my_zone_apples, otherwise nearest_respawning_apple.”

```

374 1 def voronoi_zones(env):
375 2     """Multi-source BFS Voronoi over walkable cells.
376 3     Returns (row, col) -> agent_id; ties broken by lower agent_id."""
377 4     queue, visited = deque(), {}
378 5     for a_id in range(env.n_agents):
379 6         if int(env.agent_timeout[a_id]) == 0:
380 7             ar = int(env.agent_pos[a_id][0]); ac = int(env.agent_pos[a_id][1])
381 8             queue.append((ar, ac, a_id))
382 9             visited[(ar, ac)] = a_id
383 10    while queue:
384 11        r, c, a_id = queue.popleft()
385 12        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
386 13            nr, nc = r + dr, c + dc
387 14            if 0 <= nr < env.height and 0 <= nc < env.width \
388 15                and not env.walls[nr, nc] and (nr, nc) not in visited:
389 16                visited[(nr, nc)] = a_id
390 17                queue.append((nr, nc, a_id))
391 18    return visited
392
393 19
394 20 def nearest_respawning_apple(env, agent_id, zones=None, max_timer=10):
395 21     """Nearest dead apple in MY zone respawning within max_timer steps.
396 22     Returns (row, col) of best wait spot, or None."""
397 23     if zones is None: zones = voronoi_zones(env)
398 24     best_pos, best_t, best_d = None, max_timer + 1, float('inf')
399 25     ar = int(env.agent_pos[agent_id][0]); ac = int(env.agent_pos[agent_id][1])
400 26     for i in range(env.n_apples):

```

```

401 27         if not env.apple_alive[i]:
402 28             pos = (int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
403 29             if zones.get(pos) == agent_id:
404 30                 t = int(env.apple_timer[i])
405 31                 if t <= max_timer:
406 32                     d = abs(pos[0] - ar) + abs(pos[1] - ac)
407 33                     if t < best_t or (t == best_t and d < best_d):
408 34                         best_pos, best_t, best_d = pos, t, d
409 35
410 return best_pos

```

The choice of helper depends on the dilemma’s geometry: row-bands suffice when fairness is the main concern (Cleanup maximin) but full BFS-Voronoi is needed when wall-aware territory ownership matters (Gathering). The researcher does not fix this in advance — it picks the simpler primitive whenever it works.

D.2 System prompt p : from neutral framing to strategic briefing

The unmodified system prompt is a neutral API description: “Write a policy that maximizes per-agent reward.” Under maximin optimization, the researcher rewrites the prompt into a multi-section strategic briefing. The excerpt below shows the pieces it added to the Cleanup prompt in exp5; the full prompt grew from 165 to 325 lines.

Listing 3: Cleanup, $\Phi_{\min}$ – excerpts from the researcher-rewritten system prompt (exp5). The unmodified baseline contained only API documentation; everything below was added by $\mathcal{R}$.

```

420 ## CRITICAL OBJECTIVE: Rawlsian Fairness (Maximin)
421
422 Your goal is to maximize the minimum per-agent total return across all
423 agents. This is the "maximin" or Rawlsian welfare criterion.
424
425 This means: it is NOT enough for the *average* agent to do well. The
426 worst-off agent must do as well as possible. A policy where 8 agents
427 each earn +200 but 2 agents each earn -100 is TERRIBLE (maximin = -100).
428
429 Key implication: cleaning costs (-1 per CLEAN action) must be shared
430 equitably among ALL agents. If you assign fixed "cleaner" roles, those
431 agents will accumulate large negative rewards and destroy your maximin score.
432
433 ## Strategy for Maximin: ROLE ROTATION + APPLE ZONING
434
435 The optimal strategy for maximin involves:
436 1. Shared cleaning duty: ALL agents take turns cleaning using a rotation
437    schedule based on 'agent_id' and 'env._step_count'. For example:
438    'is_my_cleaning_turn = (agent_id + env._step_count // SHIFT) % env.n_agents < NUM_CLEANERS'
439    where SHIFT is -50 steps and NUM_CLEANERS is 2-3.
440 2. When NOT your turn: collect apples from YOUR ZONE using
441    'get_my_apples(env, agent_id)' -- prevents all gatherers competing
442    for the same nearest apple.
443 3. NEVER use BEAM (action 6): it costs -1 and causes -50 to the target.
444 4. Keep waste density low: aim for -2-3 active cleaners at any time.
445
446 [... API documentation, helper documentation, working example ...]
447
448 IMPORTANT:
449 - NEVER assign permanent cleaning roles to specific agents -- this kills maximin.
450 - Use env._step_count for time-based role rotation so all agents share cleaning duty.
451 - NEVER use BEAM (action 6) -- it destroys both agents' rewards.
452

```

Two things stand out. First, the researcher writes the formula it expects $\mathcal{M}$ to use almost verbatim — `(agent_id + env._step_count // SHIFT) % env.n_agents < NUM_CLEANERS` — and the policy in Listing 6 uses essentially this template. Second, the researcher *teaches by counter-example*: the explicit “8 agents earn +200, 2 earn -100, maximin = -100” makes the failure mode of Φ_U -style static roles (Listing 5) concretely visible to $\mathcal{M}$. This is the information-design move predicted by the mechanism-design framing in Section 3.

The Gathering prompt evolves along a different axis (no maximin, no rotation). Its researcher-added “Key Strategic Insights” section instead emphasizes (i) *self-play implies never beam* and (ii) *Voronoi partition each step*, exactly matching what the policy in Listing 8 implements.

463 D.3 Feedback ϕ : adaptive diagnostics

464 The baseline feedback function ([1], dense mode) shows reward + four social metrics with defini-
 465 tions and stops there. The researcher turns it into a state machine: it inspects the latest metrics and
 466 conditionally injects different hints. Two different runs produced two qualitatively different diagnos-
 467 tics — the same metric ($\bar{r}_k$ or $\min_i R_i$) is repurposed to either *stabilize* a working policy or *redirect*
 468 a failing one.

Listing 4: Adaptive feedback diagnostics. *Top*: efficiency-optimized run (exp1) – a stability guard prevents iter-3 regression once U is high. *Bottom*: maximin-optimized run (exp5) – two fairness diagnostics that fire when the worst-off agent loses reward.

```

469 1 # --- Cleanup, Phi_U feedback (exp1, pipeline/feedback.py final) ---
470 2 last_eff = history[-1]["metrics"]["efficiency"]
471 3 if last_eff >= 2.5:
472 4     parts.append(
473 5         """CRITICAL -- DO NOT REGRESS!: The current policy achieves high "
474 6         "efficiency (>=2.5). You MUST output a policy that is nearly identical "
475 7         "to the current one. Copy the current policy code and make AT MOST "
476 8         "one small targeted improvement (e.g., adjust a single numeric "
477 9         "threshold). If you are not confident a change will help, output the "
478 10        "current policy UNCHANGED. A regression here means the run fails.")
479 11
480 12 # --- Cleanup, Phi_min feedback (exp5, pipeline/feedback.py final) ---
481 13 last_maximin = history[-1]["metrics"].get("maximin", 0)
482 14 last_avg = history[-1]["reward_avg"]
483 15 if last_maximin < 0:
484 16     parts.append(
485 17         f"""FAIRNESS ALERT!: maximin={last_maximin:.1f} is NEGATIVE. "
486 18         f"The worst-off agent lost reward overall while average was {last_avg:.1f}. "
487 19         "This means cleaning duties are NOT shared equitably. "
488 20         "Use time-based role rotation (env._step_count) so ALL agents share "
489 21         "cleaning costs equally. NEVER assign permanent cleaner roles.")
490 22 elif last_maximin < last_avg * 0.5:
491 23     parts.append(
492 24         f"""FAIRNESS WARNING!: maximin={last_maximin:.1f} is much lower than "
493 25         f"average={last_avg:.1f}. The gap suggests unequal cleaning burden. "
494 26         "Ensure ALL agents rotate through cleaning duty.")

```

497 The two diagnostics are written by independent researcher runs but converge on a common pattern:
 498 *thresholded interventions on a primary metric*. They are also a common cause of the low run-to-
 499 run variance reported in Finding 1: when a working policy lands in the high- U basin, the stability
 500 guard prevents the LLM from over-refining and tipping out of it (the $K=3$ regressions visible in
 501 Section 4’s “Common failure modes” (4)), and when an unfair policy lands in the negative-maximin
 502 region, the alert injects exactly the rotation hint that recovers it.

503 D.4 Iteration logic ι : per-condition hyperparameters

504 Although ι has the smallest source surface, the researcher’s edits to it explain a meaningful fraction
 505 of the variance reduction noted in Finding 1. Table 7 summarizes the values shipped in the final
 506 commit of each best-performing run.

Table 7: Iteration-logic (ι) values in the final commit of selected runs. K = inner-loop iterations, $|S|$ = evaluation seeds, “thinking” = extended-thinking token budget passed to $\mathcal{M}$. Baseline ([1]) is $K=3$, $|S|=5$, thinking 16k.

Run	Game / objective	K	$ S $	thinking	Researcher’s stated rationale
exp1 (Gemini)	Cleanup, Φ_U	3	5	16k	default; stability comes from feedback hint
exp4 (Sonnet)	Cleanup, Φ_U	3	5	32k	“Sonnet was over-refining at 16k thinking”
exp5 (Gemini)	Cleanup, $\Phi_{\min}$	2	12	16k	“ $K=2$ avoids iter-3 regression; 12 seeds for variance control”
exp7 (Sonnet)	Cleanup, $\Phi_{\min}$	3	5	10k	“reduced thinking from 16k – Sonnet was generating runaway code at 16k”
gather-exp3 (Gemini)	Gathering, Φ_U	3	5	16k	default; Gathering converges in $K \leq 3$

507 The maximin Gemini run is the most informative case: the researcher discovered that with $K=3$
 508 and $|S|=5$, identical configurations could yield $\min_i R_i=295$ on one set of seeds and $\min_i R_i=100$
 509 on another (this is the “variance exposure” failure mode of Section 4’s common failures). It then
 510 walked $|S|$ from 5 $\rightarrow$ 8 $\rightarrow$ 12 over three outer iterations until the seed-averaged signal was stable
 511 enough that the policy LLM stopped chasing noise. The policy in Listing 6 is sampled at $|S|=12$.

D.5 Cross-artifact observations

- *Helpers do the algebra; prompts pick the strategy class.* The researcher uses helpers (1, 2) to put non-trivial primitives at $\mathcal{M}$'s fingertips, and the prompt (3) to tell $\mathcal{M}$ which primitive to compose. Neither is sufficient alone — prompts without supporting helpers produced policies that “mention rotation” but failed to implement it; helpers without prompt updates often went unused.
- *Feedback is the only adaptive component.* p , $\mathcal{H}$, and ι are static within a run; ϕ is the only place where the researcher gets to change what $\mathcal{M}$ sees during the inner loop, and it uses thresholded diagnostics (Listing 4) to do so. This is what mostly drives the run-to-run variance reduction.
- *The same artifact has different content under each objective.* The Cleanup prompt under Φ_U omits “rotation” and “maximin” entirely; the same prompt under $\Phi_{\min}$ devotes its entire opening section to it. The configuration c is genuinely a function of the welfare objective Φ , not a fixed pipeline parameterized by it — consistent with the information-design framing of Section 3.
- *The researcher’s edits are short and targeted.* The full diff between the dense-feedback baseline and the best maximin Gemini configuration totals ~ 300 added lines spread over four files; ~ 80 of them appear in the listings above. The remaining content is API documentation and worked examples, included for completeness but not strategically novel.

E Selected Generated Policies

The code excerpts below are verbatim outputs of the policy synthesizer $\mathcal{M}$, taken from the final inner-loop iteration of the best-performing run in each condition. To fit the page, we elide repeated boilerplate (BFS scaffolding, fallback branches) with $\dots$ and add commentary in [brackets]; nothing else has been edited. The listings illustrate the qualitative differences highlighted in Findings 2–4 of Section 4: *static* vs. *rotating* cleaning roles in Cleanup, and the *Voronoi* + *respawn-timer* structure in Gathering. All four policies were independently generated by the LLM under different seeds, models, and outer-loop runs.

E.1 Cleanup, Φ_U (Gemini): static interleaved roles + dynamic threshold

Best run: $U=3.25$, $E=0.61$, $\min_i R_i = -182$. The synthesizer assigns *permanent* cleaning roles via $\text{agent_id} \bmod n < \tau$, where the threshold τ depends smoothly on the current waste fraction. Higher pollution recruits more cleaners, but the *identity* of each cleaner is fixed — agents $0\text{--}\tau-1$ always pay the cost. This drives U to its ceiling but leaves equality at 0.61 and pushes the worst-off agent to $\min_i R_i = -182$.

Listing 5: Cleanup, Φ_U – Gemini 3.1 Pro. Static role assignment with waste-tiered threshold (verbatim, abridged).

```

1 def policy(env, agent_id) -> int:
2     if int(env.agent_timeout[agent_id]) > 0:
3         return 7 # STAND while removed
4
5     waste_frac = waste_fraction(env)
6     n_agents = env.n_agents
7
8     # [Smooth dynamic threshold: more pollution -> more cleaners]
9     if waste_frac > 0.35: threshold_cleaners = n_agents
10    elif waste_frac > 0.25: threshold_cleaners = int(n_agents * 0.7)
11    elif waste_frac > 0.15: threshold_cleaners = int(n_agents * 0.5)
12    elif waste_frac > 0.05: threshold_cleaners = int(n_agents * 0.3)
13    else: threshold_cleaners = max(1, int(n_agents * 0.2))
14
15    # [STATIC role: cleaners are ALWAYS the lowest-id agents -- no rotation]
16    is_cleaner = (agent_id % n_agents) < threshold_cleaners
17
18    if is_cleaner:
19        best_orient, n_waste = find_best_clean_orientation(env, agent_id)
20        if n_waste >= 1:
21            cur_orient = int(env.agent_orient[agent_id])
22            if best_orient == cur_orient: return 8 # CLEAN
23            if (cur_orient + 1) % 4 == best_orient: return 5 # ROTATE_RIGHT
24            elif (cur_orient - 1) % 4 == best_orient: return 4 # ROTATE_LEFT
25            else: return 4
26        # [Competitor-aware BFS toward waste in own row band]
27        c_idx = agent_id % n_agents
28        target_r = int(((c_idx + 0.5) / threshold_cleaners) * env.height)

```



```

571 29         # ... pick waste cell minimizing dist + |row - target_r| + 50 *
572         # closer_cleaners
573 30         # ... bfs_toward and direction_to_action
574 31     else:
575 32         # GATHERER: row-banded sweep, competitor-aware
576 33         # assigned_row = ((g_idx + 0.5) / n_gatherers) * env.height
577 34         # ... pick alive apple minimizing dist + |row - assigned_row| + 50 *
578         # closer_gatherers
579 35     ...

```

581 E.2 Cleanup, $\Phi_{\min}$ (Gemini): time-rotated, geographically interleaved roles

582 Best run: $U=3.19$, $E=0.98$, $\min_i R_i=296$. Switching the outer objective from U to $\min_i R_i$ flips
583 the role-assignment scheme entirely: `role_idx` now depends on `agent_id + step/50`, so every
584 agent rotates through cleaner and gatherer duty in a 50-step cycle. The cleaner indices $\{1, 4, 8\}$ are
585 spatially *interleaved* along the river (rather than contiguous) so that whoever is on duty has a short
586 walk to their assigned river slice. This single change accounts for the $E=0.55 \rightarrow 0.98$ jump and the
587 $\min_i R_i=-182 \rightarrow 296$ swing reported in Table 3, with negligible efficiency loss.

Listing 6: Cleanup, $\Phi_{\min}$ – Gemini 3.1 Pro. Time-rotated roles with interleaved geography (verbatim, abridged).

```

588 1 def policy(env, agent_id) -> int:
589 2     if int(env.agent_timeout[agent_id]) > 0:
590 3         return 7 # STAND while removed
591 4
592 5     aid = int(agent_id)
593 6     step = int(env._step_count)
594 7     n = int(env.n_agents)
595 8
596 9     # [TIME ROTATION: every 50 steps each agent's role index advances]
597 10    if n == 10:
598 11        role_idx = (aid + step // 50) % 10
599 12        # [Cleaner indices {1,4,8} are SPATIALLY INTERLEAVED, not contiguous,
600 13        # so each on-duty cleaner has a short walk to its river slice]
601 14        if role_idx == 1: role_type, zone_idx = 'C', 0
602 15        elif role_idx == 4: role_type, zone_idx = 'C', 1
603 16        elif role_idx == 8: role_type, zone_idx = 'C', 2
604 17        else:
605 18            role_type = 'G'
606 19            g_map = {0:0, 2:1, 3:2, 5:3, 6:4, 7:5, 9:6}
607 20            zone_idx = g_map[role_idx]
608 21            num_cleaners, num_gatherers = 3, 7
609 22
610 23    if role_type == 'C':
611 24        # [Each on-duty cleaner owns one horizontal slice of the river]
612 25        row_min = int( zone_idx * env.height / num_cleaners)
613 26        row_max = int((zone_idx + 1) * env.height / num_cleaners) - 1
614 27
615 28        # [Score every (row, col, orientation) in the slice by beam yield;
616 29        # fire if facing >=2 waste, else move to a >=2-yield position]
617 30        best_positions, good_positions = set(), set()
618 31        for r in range(row_min, row_max + 1):
619 32            for c in range(10):
620 33                if env.walls[r, c] or env.waste[r, c]: continue
621 34                y_max = max(get_clean_yield(env, r, c, o) for o in range(4))
622 35                if y_max >= 2: best_positions.add((r, c))
623 36                if y_max >= 1: good_positions.add((r, c))
624 37            # ... rotate / CLEAN / bfs_to_target_set as appropriate
625 38    else:
626 39        # GATHERER: collect apples ONLY inside the rotating row band
627 40        # row_min/row_max for num_gatherers slices ...
628 41    ...
629 42

```

631 E.3 Cleanup, $\Phi_{\min}$ (Sonnet): independent rediscovery of duty rotation

632 Best run: $U=2.93$, $E=0.83$, $\min_i R_i=154$. Run with a different policy LLM (Sonnet 4.6) and on
633 a different outer-loop trajectory, the synthesizer arrives at the *same* structural insight as the Gemini
634 maximin run: a phase counter (`agent_id + step/50`) mod n rotates which 2 of the 10 agents clean
635 at any given time, and a *separate* 200-step zone counter rotates which 5-row band each collector
636 sweeps. The two rotation periods (50 and 200 steps) ensure every agent visits every cleaning slot
637 and every apple zone within one episode — a structural fairness invariant the researcher discovered
638 without any explicit instruction to “rotate.”

Listing 7: Cleanup, $\Phi_{\min}$ – Sonnet 4.6. Phase-rotated cleaning + zone-rotated collection (verbatim, abridged).

```

639 1 def policy(env, agent_id) -> int:
640 2     if int(env.agent_timeout[agent_id]) > 0:
641 3         return 7 # STAND while removed
642 4
643 5     n = env.n_agents
644 6     step = env._step_count
645 7     wf = waste_fraction(env)
646 8
647 9     # --- ROLE ASSIGNMENT: fair 2-cleaner rotation ---
648 10    # [50-step phases x 20 phases / episode -> each agent cleans 4 phases (~20%)]
649 11    phase = step // 50
650 12    cleaner_rank = (agent_id + phase) % n # [cycles 0..9 fairly]
651 13    is_cleaner = cleaner_rank < 2 # [exactly 2 cleaners per phase]
652 14
653 15    # [Emergency override: all agents clean when waste crosses the spawn cliff]
654 16    in_emergency = wf >= 0.40
655 17    if in_emergency: is_cleaner = True
656 18
657 19    # [River split: cleaner_slot 0 -> top half, 1 -> bottom half]
658 20    cleaner_slot = (agent_id % 2) if in_emergency else cleaner_rank
659 21
660 22    if is_cleaner and wf > 0.04:
661 23        # [Scan 4 orientations from current position; if best beam shot
662 24        #   covers >=3 waste cells, fire; else step to a strictly better neighbor]
663 25        # ... (best_o, best_cnt) = max over 4 directions
664 26        # ... if best_cnt >= 3: rotate to best_o, then CLEAN
665 27        # ... else: scan 4 adjacent cells x 4 orientations for higher yield
666 28        # ... if no waste in beam range: bfs to waste_set in OWN river half,
667 29        #   fall back to whole river
668 30
669 31    # --- COLLECTOR LOGIC: 5-zone rotating apple collection ---
670 32    # [5 zones x 200 steps = 1000 steps -> each agent visits every zone once;
671 33    # 2 agents share each zone at any time, reducing competition 10 -> 2]
672 34    bounds = zone_boundaries(env, 5)
673 35    zone_phase = step // 200
674 36    zone = (agent_id + zone_phase) % 5
675 37    z_start, z_end = bounds[zone], bounds[zone + 1]
676 38
677 39    zone_apples = {(int(env._apple_pos[i][0]), int(env._apple_pos[i][1]))
678 40                  for i in range(env.n_apples) if env.apple_alive[i]
679 41                  and z_start <= int(env._apple_pos[i][0]) < z_end}
680 42    # ... bfs_to_target_set(zone_apples), else bfs_nearest_apple as fallback
681 43    return 7
682

```

684 E.4 Gathering (Gemini): wall-aware Voronoi + spatiotemporal targeting

685 Best run: $U=2.47$, $E=0.98$, $\min_i R_i=580$. In Gathering, where costs are symmetric, no role differentiation is needed: the entire optimization is spatial and temporal. The synthesizer (i) partitions cells into BFS-Voronoi territories owned by individual agents, (ii) runs a single centralized BFS from its own position to score every reachable cell with exact wall-aware distances, and (iii) for both alive and respawning apples in its territory, computes the *earliest collection time* $\max(\text{walk_dist}, \text{respawn_timer})$ and targets the minimum. When the agent must wait on a dead apple, it steps onto a *non-spawn* adjacent cell rather than blocking a respawn point. This single sort key — $\max(\text{distance}, \text{timer})$ — subsumes both “go to nearest apple” and “camp respawn” as special cases.

Listing 8: Gathering – Gemini 3.1 Pro. Voronoi territories + spatiotemporal priority targeting (verbatim, abridged).

```

694 1 def policy(env, agent_id) -> int:
695 2     if int(env.agent_timeout[agent_id]) > 0:
696 3         return 7
697 4
698 5     r, c = int(env.agent_pos[agent_id][0]), int(env.agent_pos[agent_id][1])
699 6     orient = int(env.agent_orient[agent_id])
700 7
701 8     # [Wall-aware Voronoi over the gridworld; spawn_points = all apple cells]
702 9     zones = voronoi_zones(env)
703 10    spawn_points = {(int(p[0]), int(p[1])) for p in env._apple_pos}
704 11
705 12    # [Single centralized BFS: exact distance + first move to every reachable cell]
706 13    distances, first_moves = {}, {}
707 14    q = deque([(r, c, 0, None)])
708

```

```

709 15     visited = {(r, c)}
710 16     while q:
711 17         cr, cc, d, fm = q.popleft()
712 18         distances[(cr, cc)] = d
713 19         first_moves[(cr, cc)] = fm
714 20         for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
715 21             nr, nc = cr + dr, cc + dc
716 22             if 0 <= nr < env.height and 0 <= nc < env.width and not env.walls[nr,
717 23                 nc]:
718 24                 if (nr, nc) not in visited:
719 25                     visited.add((nr, nc))
720 26                     q.append((nr, nc, d + 1, fm if fm is not None else (dr, dc)))
721 27
722 28     # [Both alive AND respawning apples in MY zone, with their respawn timer]
723 29     my_zone_spawns = []
724 30     for i in range(env.n_apples):
725 31         pr, pc = int(env._apple_pos[i][0]), int(env._apple_pos[i][1])
726 32         if zones.get((pr, pc)) == agent_id and (pr, pc) in distances:
727 33             my_zone_spawns.append((pr, pc, int(env.apple_timer[i]), distances[(pr,
728 34                 pc)]))
729 35
730 36     # [SPATIOTEMPORAL PRIORITY: earliest collection time first.
731 37     # score = max(walk_distance, respawn_timer); ties broken by sooner timer]
732 38     my_zone_spawns.sort(key=lambda x: (max(x[3], x[2]), x[2], x[3]))
733 39
734 40     for pr, pc, timer, dist in my_zone_spawns:
735 41         if timer == 0:
736 42             # [Alive apple: walk straight to it]
737 43             if dist == 0: return 7
738 44             fm = first_moves[(pr, pc)]
739 45             return direction_to_action(fm[0], fm[1], orient)
740 46         else:
741 47             # [Dead apple: camp on safe adjacent cell]
742 48             # ... pick nearest neighbor (nr,nc) that is NOT in spawn_points,
743             # ... navigate there and STAND while waiting for respawn
744             # [Global poaching fallback if zone is empty: nearest alive apple anywhere]
745             ...

```

746 E.5 Cross-condition observations

747 Several patterns are visible across these four listings (and confirmed by inspecting the remaining 8
748 runs not shown):

- 749 • *Role assignment encodes the welfare objective.* Static $\text{agent_id} < \tau$ (Listing 5) maximizes
750 U but harms $\min_i R_i$. Time-rotated $(\text{agent_id} + \text{step} // T) \% n$ (Listings 6, 7) maximizes
751 $\min_i R_i$ at $\leq 1\%$ efficiency cost (Gemini).
- 752 • *Convergent rediscovery across LLMs.* Gemini and Sonnet, on independent runs, both arrive at
753 the $(\text{id} + \text{phase}) \% n$ duty-rotation idiom with similar phase lengths ($T \approx 50$ steps) under
754 maximin — and both omit it under efficiency.
- 755 • *Game structure dictates strategy class.* Cleanup policies are dominated by role-assignment logic
756 (cleaner vs. gatherer); Gathering policies are dominated by territory partitioning and respawn-
757 timer reasoning, with no role differentiation at all. The researcher does not need to be told which
758 class of strategy to write.
- 759 • *Earliest-collection-time as a unifying score.* The Gathering policy’s $\max(\text{walk}, \text{timer})$ key (List-
760 ing 8) is structurally identical across the 4 Gathering runs (both Gemini and Sonnet), differing
761 only in how the Voronoi diagram is computed — Manhattan approximation, fully BFS-based, or
762 cached helper.